

# Software Development Life Cycle (SDLC)

Library Management System

**Uttar Pradesh State Tax Training & Research Institute (UPSTTRI)**

Version 1.1.2 · Document Revision 1.0

Date: June 2026

Flutter Desktop

Node.js + Express

MySQL

REST API

JWT Auth

# SDLC Document

A complete walkthrough of how the UPSTTRI Library Management System was planned, designed, built, tested, deployed, and is maintained.

## Table of Contents

---

1. Introduction & Document Purpose
2. Project Overview
3. SDLC Model Selected & Justification
4. Phase 1 — Requirement Analysis & Planning
5. Phase 2 — System Design
6. Phase 3 — Implementation / Development
7. Phase 4 — Testing & Quality Assurance
8. Phase 5 — Deployment & Release
9. Phase 6 — Maintenance & Support
10. Risk Management
11. Project Timeline & Milestones
12. Conclusion

# 1. Introduction & Document Purpose

This document describes the Software Development Life Cycle (SDLC) followed for the design and delivery of the **UPSTTRI Library Management System**. The SDLC is a structured framework that defines the activities performed at each stage of software development, ensuring the final product is reliable, maintainable, secure, and aligned with stakeholder expectations.

The purpose of this document is to:

- Provide a single reference describing how the system moves from concept to a maintained production application.
- Capture the requirements, design decisions, technology choices, and quality controls applied at each phase.
- Serve as onboarding material for new developers and as an audit trail for stakeholders at the institute.

# 2. Project Overview

The Library Management System is a production-ready **Windows desktop application** built for the Uttar Pradesh State Tax Training & Research Institute. It digitises and automates the core operations of a library: cataloguing books, managing members, issuing and returning books, tracking overdue items, and generating analytical reports.

## 2.1 Goals

- Replace manual register-based book tracking with a fast, searchable digital catalogue.
- Automate due-date calculation, overdue detection, and fine computation.
- Provide administrators with real-time dashboards, reports, and exportable data.
- Enforce strong security suitable for a government institution.

## 2.2 Stakeholders

| Stakeholder                        | Role / Interest                                                   |
|------------------------------------|-------------------------------------------------------------------|
| Institute Administration           | Project sponsor; defines policy, approves budget and rollout.     |
| Librarian / Admin User             | Primary day-to-day user managing books, members, and circulation. |
| Members (Students, Faculty, Staff) | Beneficiaries whose records and loans are managed in the system.  |
| Development Team                   | Designs, builds, tests, and maintains the software.               |
| IT / Systems Support               | Handles deployment, database hosting, and backups.                |

## 2.3 Technology Stack

| Layer            | Technology                                                 |
|------------------|------------------------------------------------------------|
| Frontend         | Flutter 3.10+ · Material 3 · Provider state management     |
| Backend          | Node.js · Express 4 · REST API (TypeScript build pipeline) |
| Database         | MySQL 8.0+ with connection pooling and auto-migrations     |
| Authentication   | JWT (HS256) · bcrypt · Helmet · express-rate-limit         |
| Charts & Reports | FL Chart · pdf + printing packages                         |
| File Handling    | Multer (backend) · file_picker (Flutter) · CSV/XLSX        |
| Packaging        | Inno Setup installer for Windows 10/11 (64-bit)            |

### 3. SDLC Model Selected & Justification

The project adopted an **Iterative & Incremental model** with Agile practices. The system was delivered in successive increments — authentication first, then book management, member management, circulation, reporting, and finally polish features such as notifications, backup/restore, and PDF export.

#### 3.1 Why Iterative / Incremental?

- **Evolving requirements:** Library workflows and member categories were refined as stakeholders saw working software, which suits iterative feedback better than a rigid Waterfall plan.
- **Early usable value:** Each increment produced a working module that could be demonstrated and validated.
- **Risk reduction:** Integration risks (Flutter  $\rightleftharpoons$  Node  $\rightleftharpoons$  MySQL) were discovered early because a thin end-to-end slice was built first.
- **Continuous testing:** Automated Jest API tests and a CI workflow allowed each increment to be validated before the next began.

Although the model is iterative, every increment still passed through all classic SDLC phases — requirements, design, implementation, testing, and integration — in miniature. The sections below describe these phases as applied across the project.

### 4. Phase 1 — Requirement Analysis & Planning

In this phase the team gathered, analysed, and documented what the system must do and the constraints it must operate under.

#### 4.1 Functional Requirements

| ID   | Requirement                                                                                                               |
|------|---------------------------------------------------------------------------------------------------------------------------|
| FR-1 | Admin authentication via username/password with secure session tokens and forced password change on first login.          |
| FR-2 | Full CRUD for books including ISBN, author, category, rack number, publisher, year, cover image, and multi-copy tracking. |
| FR-3 | Full CRUD for members with profile photos, member types, and active/inactive status.                                      |
| FR-4 | Issue and return books with automatic due-date calculation and fine computation for overdue items.                        |
| FR-5 | Real-time dashboard with KPIs, charts, actionable alerts, and a recent-activity feed.                                     |
| FR-6 | Reports (issued, overdue, popular books, active members, category and monthly stats) exportable to PDF/Excel.             |

|       |                                                                                  |
|-------|----------------------------------------------------------------------------------|
| FR-7  | In-app notifications for overdue, due-soon, new-book, and system events.         |
| FR-8  | Global search and advanced multi-field search across books, members, and issues. |
| FR-9  | Bulk import (CSV/XLSX), bulk delete, and CSV/JSON export of catalogue data.      |
| FR-10 | Full JSON backup and restore of books, members, and issues.                      |

## 4.2 Non-Functional Requirements

| Category        | Requirement                                                                                       |
|-----------------|---------------------------------------------------------------------------------------------------|
| Security        | JWT auth, bcrypt hashing, parameterised SQL, Helmet headers, CORS whitelist, rate limiting.       |
| Performance     | Indexed hot-path queries; paginated list endpoints; sub-second list loads for typical catalogues. |
| Usability       | Premium Material 3 UI, dark/light themes, responsive layouts, Hindi/Devanagari support in PDFs.   |
| Reliability     | Connection pooling, auto-migrations on boot, graceful error handling.                             |
| Portability     | Single-machine Windows desktop deployment with bundled backend.                                   |
| Maintainability | Modular Flutter widgets/providers and a documented REST API.                                      |

## 4.3 Feasibility & Planning

- **Technical feasibility:** Flutter desktop + Node/MySQL is a proven, free, open-source stack runnable on existing institute hardware.
- **Operational feasibility:** A single admin role keeps training and operations simple for library staff.
- **Economic feasibility:** No licensing cost; runs on commodity Windows machines.

## 5. Phase 2 — System Design

The design phase translated requirements into an architecture, data model, and interface design.

### 5.1 Architecture Overview

The system follows a three-tier client-server architecture:

**Presentation Tier** — Flutter Windows desktop app. Uses the Provider pattern for state management with dedicated providers for auth, books, members, issues, reports, dashboard, notifications, search, and theme.

**Application Tier** — Node.js + Express REST API. Handles authentication, business rules (due dates, fines, availability), validation, file uploads, and report aggregation. On Windows the app can auto-launch the bundled backend process.

**Data Tier** — MySQL 8 database accessed through a connection pool. Runtime migrations add performance indexes on boot.

### 5.2 Data Model (Core Entities)

| Table                | Purpose                                                                        |
|----------------------|--------------------------------------------------------------------------------|
| users                | Admin accounts; password hash, role, forced password-change flag.              |
| books                | Catalogue records with copies, status, cover image, and indexed search fields. |
| book_categories      | Genre/category master list.                                                    |
| members              | Library members with type, photo, membership/expiry dates, active flag.        |
| member_categories    | Borrowing limits and loan periods per category.                                |
| issues               | Circulation records: issue/due/return dates and status.                        |
| notifications        | In-app alerts with type, read state, and related entity references.            |
| dashboard_settings   | Per-user widget layout and visibility.                                         |
| book_recommendations | Tracks recommendation scores from borrowing history.                           |

### 5.3 Key Relationships

- An `issue` links one `book` to one `member` (foreign keys with cascade delete).
- A `member` optionally belongs to a `member_category` defining its loan limits.
- `notifications` and `dashboard_settings` reference a `user`.

## 5.4 API Design

The backend exposes a versioned REST API under `/api`. All endpoints except health and login require a `Bearer` token. Resources are grouped as Auth, Books, Members, Issues, Categories, Dashboard, Reports, Notifications, Search, Data Operations, and Uploads. List endpoints support pagination and filtering.

## 5.5 Security Design

| Layer               | Measure                                                                                           |
|---------------------|---------------------------------------------------------------------------------------------------|
| Authentication      | JWT HS256, bcrypt (10 rounds), admin-only enforcement server-side.                                |
| Brute force         | Login limiter: 15 attempts per 15-minute window per IP.                                           |
| SQL                 | 100% parameterised queries — no string interpolation.                                             |
| Transport / headers | Helmet (CSP, X-Frame, HSTS), CORS whitelist.                                                      |
| File uploads        | Extension + MIME validation, size caps, sanitised filenames.                                      |
| Token storage       | OS keychain via <code>flutter_secure_storage</code> with <code>SharedPreferences</code> fallback. |

## 6. Phase 3 — Implementation / Development

---

Development proceeded in increments, with the frontend and backend evolving together against the agreed API contract.

### 6.1 Backend Development

- Built with Express 4; routes organised by resource with shared middleware for auth, validation (express-validator), compression, logging (morgan), and rate limiting.
- TypeScript source ( `server.ts` ) compiled to `dist/server.js` ; a bundle script produces a release artifact.
- Auto-migrations create indexes and runtime-only tables on boot, so a fresh install works without manual schema steps.
- Business logic centralises due-date calculation by member category, availability counters, and overdue/fine handling.

### 6.2 Frontend Development

- Single-window desktop shell with a premium sidebar navigation and content widgets per module (dashboard, books, members, issues, reports).
- State managed through nine Provider classes; an `ApiService` wraps all HTTP calls and a `BackendService` manages the bundled backend lifecycle.
- Reusable dialogs (book, member, advanced search, backup/restore, borrow slip) and skeleton loaders for perceived performance.
- Devanagari font bundling enables correct Hindi rendering in generated PDFs.

### 6.3 Coding Standards & Version Control

- Git used for version control with a CI workflow ( `.github/workflows/test.yml` ) running automated tests.
- Secrets kept out of the repository via `.env` (with a committed `.env.example` template).
- Flutter static analysis ( `flutter analyze` ) and lint rules enforce code quality.

## 7. Phase 4 — Testing & Quality Assurance

---

Testing was continuous and layered to validate each increment before integration.

### 7.1 Test Levels

| Level      | Approach                                                                                                                                                                              |
|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Unit / API | Jest + supertest test suites under <code>backend/__tests__</code> covering auth, books, members, issues, categories, search, reports, notifications, dashboard, and health endpoints. |

|             |                                                                                                               |
|-------------|---------------------------------------------------------------------------------------------------------------|
| Integration | <code>integration_test.js</code> and smoke-test scripts exercise full request flows against a running server. |
| System      | End-to-end manual validation of Flutter UI against the live backend.                                          |
| Performance | Large-import test tooling validates bulk CSV/XLSX ingestion.                                                  |

## 7.2 Test Types Covered

- **Functional:** Each endpoint and screen validated against its requirement.
- **Security:** Auth enforcement, rate limiting, and injection-resistant restore logic verified.
- **Regression:** CI runs the Jest suite on every change to catch breakage early.
- **Usability:** Theme switching, responsiveness, and dialog flows reviewed with stakeholders.

Continuous Integration: the GitHub Actions workflow runs the backend test suite automatically, providing a quality gate before changes are merged.

## 8. Phase 5 — Deployment & Release

---

The system is packaged as a self-contained Windows desktop product.

### 8.1 Build Process

1. Backend: `npm run build:release` compiles TypeScript and bundles the server.
2. Frontend: `flutter build windows` produces the release runner.
3. Packaging: `iscc installer.iss` (Inno Setup) creates a single installer that bundles the app and backend.

### 8.2 Deployment Steps

1. Provision MySQL 8 and create the `library_management` database.
2. Load `database/schema_v2.sql` to create tables and seed defaults.
3. Configure backend `.env` with database credentials and a strong JWT secret.
4. Install the application via the generated installer; the app launches the bundled backend automatically.
5. Log in with the default admin account and complete the forced password change.

### 8.3 Release Versioning

Semantic versioning is used (current release **1.1.2**). The version is defined in `pubspec.yaml` and surfaced in the app's About section.

## 9. Phase 6 — Maintenance & Support

---

After release the system enters ongoing maintenance, which covers four standard categories:

| Type       | Activities                                                                |
|------------|---------------------------------------------------------------------------|
| Corrective | Fixing defects reported by users or surfaced by monitoring and logs.      |
| Adaptive   | Updating to new Flutter/Node/MySQL versions and OS changes.               |
| Perfective | Adding enhancements such as new reports, member types, or UI refinements. |
| Preventive | Refactoring, dependency updates, security patching, and routine backups.  |

- **Backups:** Built-in JSON backup/restore plus regular MySQL dumps protect against data loss.
- **Monitoring:** Health endpoints ( `/api/health` , `/api/health/detailed` ) confirm service and database status.
- **Support:** Issues triaged, prioritised, and released in incremental updates.

## 10. Risk Management

| Risk                       | Impact | Mitigation                                                                         |
|----------------------------|--------|------------------------------------------------------------------------------------|
| Data loss / corruption     | High   | Automated JSON backups, MySQL dumps, foreign-key integrity, transactional updates. |
| Unauthorised access        | High   | JWT auth, bcrypt, admin-only enforcement, rate limiting, secure token storage.     |
| Bulk-import errors         | Medium | Smart header mapping, validation, and dedicated large-import tests.                |
| Dependency / version drift | Medium | Pinned versions, CI test gate, periodic adaptive maintenance.                      |
| Single-machine failure     | Medium | Documented reinstall + restore procedure; portable backup files.                   |

## 11. Project Timeline & Milestones

| Milestone | Increment Focus                                                          |
|-----------|--------------------------------------------------------------------------|
| M1        | Foundation: auth, database schema, API skeleton, app shell.              |
| M2        | Book management with categories, cover images, and multi-copy tracking.  |
| M3        | Member management with types, photos, and status control.                |
| M4        | Issue/return circulation with due dates and fine calculation.            |
| M5        | Dashboard, reports, charts, and PDF/Excel export.                        |
| M6        | Notifications, search, recommendations, backup/restore, bulk operations. |
| M7        | Hardening, testing, installer packaging, and release 1.1.2.              |

## 12. Conclusion

The UPSTTRI Library Management System was delivered through a disciplined, iterative SDLC that balanced rapid stakeholder feedback with rigorous engineering practices. Each phase — from requirement analysis through design, implementation, testing, deployment, and maintenance — contributed to a secure, performant, and user-friendly product. The modular architecture and automated test coverage position the system for straightforward future enhancement and long-term maintainability.